

EGE UNIVERSITY

Faculty of Engineering - Computer Engineering Department

Computational Intelligence and Deep Learning PROJECT 1

Fuzzy Logic, Swarm Intelligence and Deep Learning Models

2025-2026 Spring Semester

1) Deep Learning and Swarm Intelligence

1.i) What is Fuzzy Logic?

Fuzzy Logic is a form of many-valued logic that deals with approximate reasoning rather than fixed and exact binary truth values. Unlike classical Boolean logic where variables must be either true (1) or false (0), fuzzy logic allows truth values to range continuously between 0 and 1, representing degrees of truth. It was first proposed by Lotfi A. Zadeh in 1965.

Fuzzy Logic is important because real-world problems rarely have crisp boundaries. For example, the concept of 'tall' is not binary - a person of 175 cm is somewhat tall, not definitively tall or not tall. Fuzzy logic captures this natural imprecision mathematically, enabling systems to reason with uncertain, vague, or incomplete information in a way that mimics human reasoning.

Fuzzy Set Operations

The three fundamental operations on fuzzy sets are:

Intersection (AND) - The minimum of membership values:

$$\mu_{A \text{ intersect } B}(x) = \min(\mu_A(x), \mu_B(x))$$

Union (OR) - The maximum of membership values:

$$\mu_{A \text{ union } B}(x) = \max(\mu_A(x), \mu_B(x))$$

Complement (NOT) - One minus the membership value:

$$\mu_{A \text{ complement}}(x) = 1 - \mu_A(x)$$

1.ii) Key Fuzzy Logic Concepts

Membership Function

A membership function maps each element of the universe of discourse to a value in $[0, 1]$, representing the degree to which that element belongs to the fuzzy set. Common types include triangular (defined by three points), trapezoidal (four points), and Gaussian (defined by mean and standard deviation). For example, a triangular membership function for 'medium temperature' might be defined as $\text{trimf}(x; 15, 25, 35)$, peaking at 25 degrees.

Fuzzy Rule

A fuzzy rule is an IF-THEN statement using linguistic variables. For example: 'IF temperature IS hot AND humidity IS high THEN fan_speed IS fast.' Rules form the knowledge base of a fuzzy inference system. The antecedent (IF part) is evaluated using fuzzy set operations, and the consequent (THEN part) is activated proportionally to the degree of truth of the antecedent.

Defuzzification

Defuzzification converts the fuzzy output set into a single crisp value. The most common method is the Centroid method (center of gravity), which computes the weighted average: $z^* = \frac{\int z * \mu(z) dz}{\int \mu(z) dz}$. Other methods include Bisector (divides area in half), Mean of Maximum (MOM), and Smallest/Largest of Maximum.

Hedge

A hedge is a linguistic modifier that adjusts a membership function. Common hedges include: 'very' (concentrates the set by squaring: $\mu_{\text{very } A}(x) = [\mu_A(x)]^2$), 'somewhat' (dilates the set by square root: $\mu_{\text{somewhat } A}(x) = \sqrt{\mu_A(x)}$), 'extremely' ($[\mu_A(x)]^3$), and 'not very' ($1 - [\mu_A(x)]^2$). Hedges allow more precise linguistic expression in fuzzy rules.

1.iii) Application Areas of Fuzzy Logic

1. Autonomous Vehicle Steering Control

Fuzzy logic controllers manage steering, braking, and acceleration in autonomous vehicles. Unlike PID controllers that require precise mathematical models, fuzzy controllers use linguistic rules like 'IF obstacle IS close AND speed IS fast THEN brake IS hard.' The advantage is robustness to sensor noise and the ability to encode expert driving knowledge directly as rules, without requiring an exact dynamic model of the vehicle.

2. Medical Diagnosis Systems

Fuzzy logic is used in medical diagnosis where symptoms have degrees of severity. A system might encode rules such as 'IF fever IS high AND cough IS moderate THEN pneumonia_risk IS elevated.' Unlike binary decision trees, fuzzy systems handle the inherent uncertainty in medical data and produce risk levels rather than binary diagnoses, which better supports clinical decision-making.

3. Smart Home HVAC Control

Fuzzy controllers optimize heating, ventilation, and air conditioning based on imprecise comfort preferences. Rules like 'IF room_temperature IS slightly_cold AND time IS morning THEN heater IS medium' capture human comfort intuition. This outperforms fixed thermostat thresholds by providing smoother control, better energy efficiency, and adaptation to subjective comfort levels.

1.iv) Fuzzy Logic Application: Student Performance Evaluation

We developed a Student Performance Evaluation System using the scikit-fuzzy library. The system evaluates students based on three inputs and produces a final grade.

Inputs: attendance_rate (0-100%), assignment_score (0-100), exam_score (0-100)
Output: final_grade (0-100)

Each input has three fuzzy sets (low, medium, high) defined with triangular membership functions. We defined rules such as:

- IF attendance IS high AND assignment IS high AND exam IS high THEN grade IS excellent
- IF attendance IS low AND exam IS low THEN grade IS poor
- IF assignment IS medium AND exam IS medium THEN grade IS average

Example computation: attendance=75, assignment=82, exam=68
The system evaluates each rule's activation strength, aggregates the outputs, and applies centroid defuzzification. The result is a final grade of approximately 65-70, reflecting a 'good' performance level.

The complete Python implementation using skfuzzy.control is provided in scripts/fuzzy_student_evaluation.py.

1.v) Comparison: ABC vs Differential Evolution

Criterion	ABC	DE
Inspiration	Honey bee foraging	Vector differences
Population	Employed/Onlooker/Sc	Single population
Phases	3 (employed,onlooker	Mutation,Crossover,S
Exploration	Scout bees (random)	Mutation (F paramete
Exploitation	Onlooker bees (greed	Crossover (CR param)
Convergence	Moderate	Fast
Parameters	Limit (abandon count	F (mutation), CR (cr
Best for	Multimodal problems	Continuous optimizat

The Artificial Bee Colony (ABC) algorithm is inspired by the foraging behavior of honey bees. It uses three groups: employed bees (exploit known food sources), onlooker bees (select promising sources based on waggle dance), and scout bees (explore randomly when a source is abandoned). ABC excels at balancing exploration and exploitation.

Differential Evolution (DE) is a population-based optimizer that creates mutant vectors by adding weighted differences of population members: $v = x_{r1} + F \cdot (x_{r2} - x_{r3})$. Crossover creates trial vectors, and selection keeps the better solution. DE is simple, fast-converging, and effective for continuous optimization problems.

Both are metaheuristic algorithms for global optimization, but ABC is better for multimodal landscapes with many local optima, while DE is preferred for continuous parameter optimization with faster convergence.

2) Machine Learning, Deep Learning and Gen AI

2.i) ROC Curve

The Receiver Operating Characteristic (ROC) curve is a graphical tool for evaluating binary classification models across all possible classification thresholds.

The x-axis represents the False Positive Rate: $FPR = FP / (FP + TN)$, which measures the proportion of actual negatives incorrectly classified as positive. The y-axis represents the True Positive Rate: $TPR = TP / (TP + FN)$, which measures the proportion of actual positives correctly identified.

The ROC curve can be understood in three visual steps:

1. Threshold variation: As we lower the classification threshold from 1 to 0, more instances are classified as positive, increasing both TPR and FPR.
2. Plotting TPR vs FPR at each threshold traces the ROC curve from (0,0) to (1,1).
3. The Area Under the Curve (AUC) summarizes overall performance: $AUC = 1.0$ means perfect classification, $AUC = 0.5$ means random (diagonal line), $AUC < 0.5$ is worse than random.

In our LSTM project, we computed ROC curves for the directional prediction task (up vs down), achieving AUC values between 0.506 and 0.524, indicating slight predictive power above random chance - which is realistic for financial markets.

ROC Curves — XAUUSD_1H

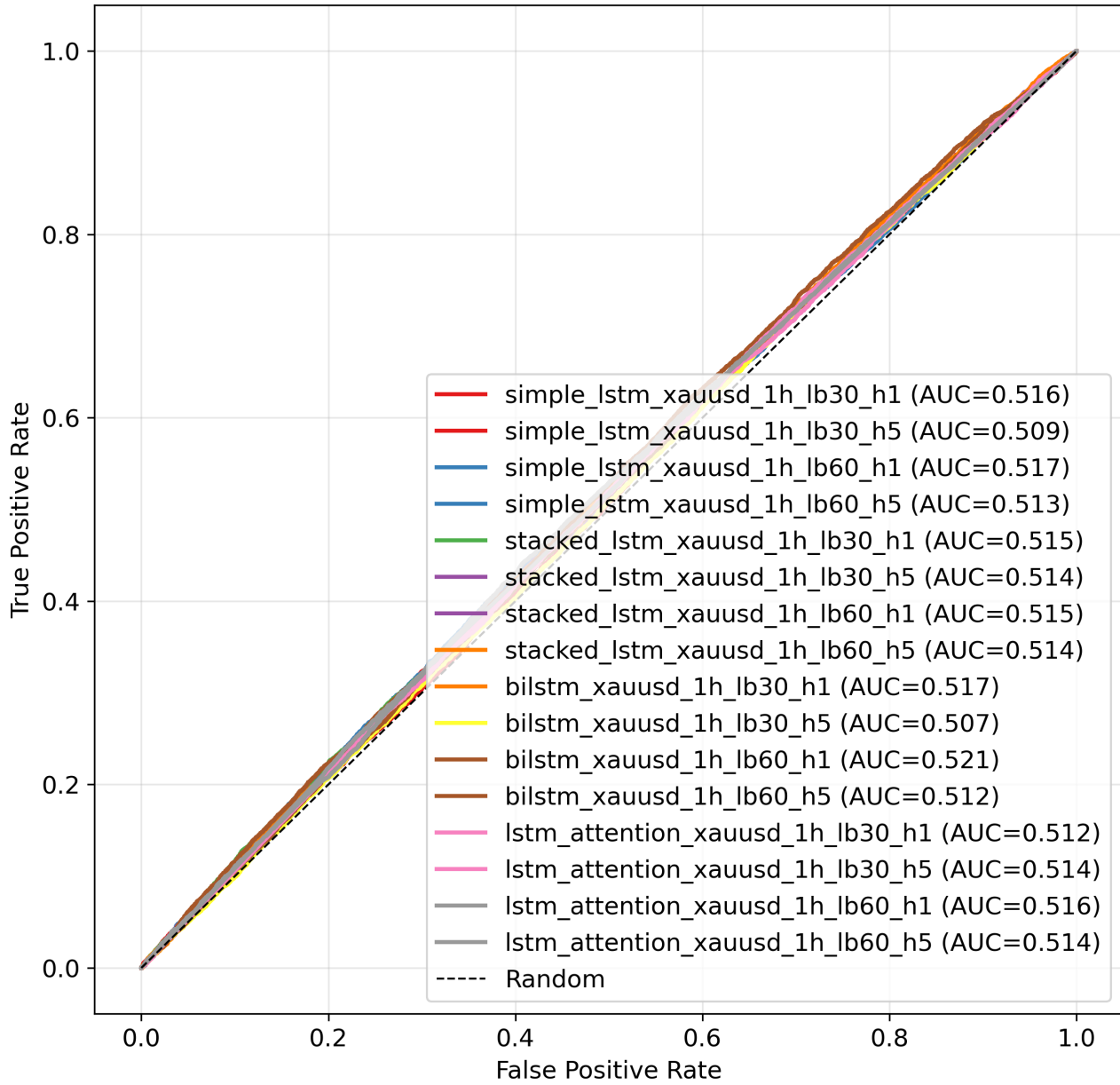


Figure: Roc Curves Xauusd 1H

ROC Curves — BTCUSD_1D

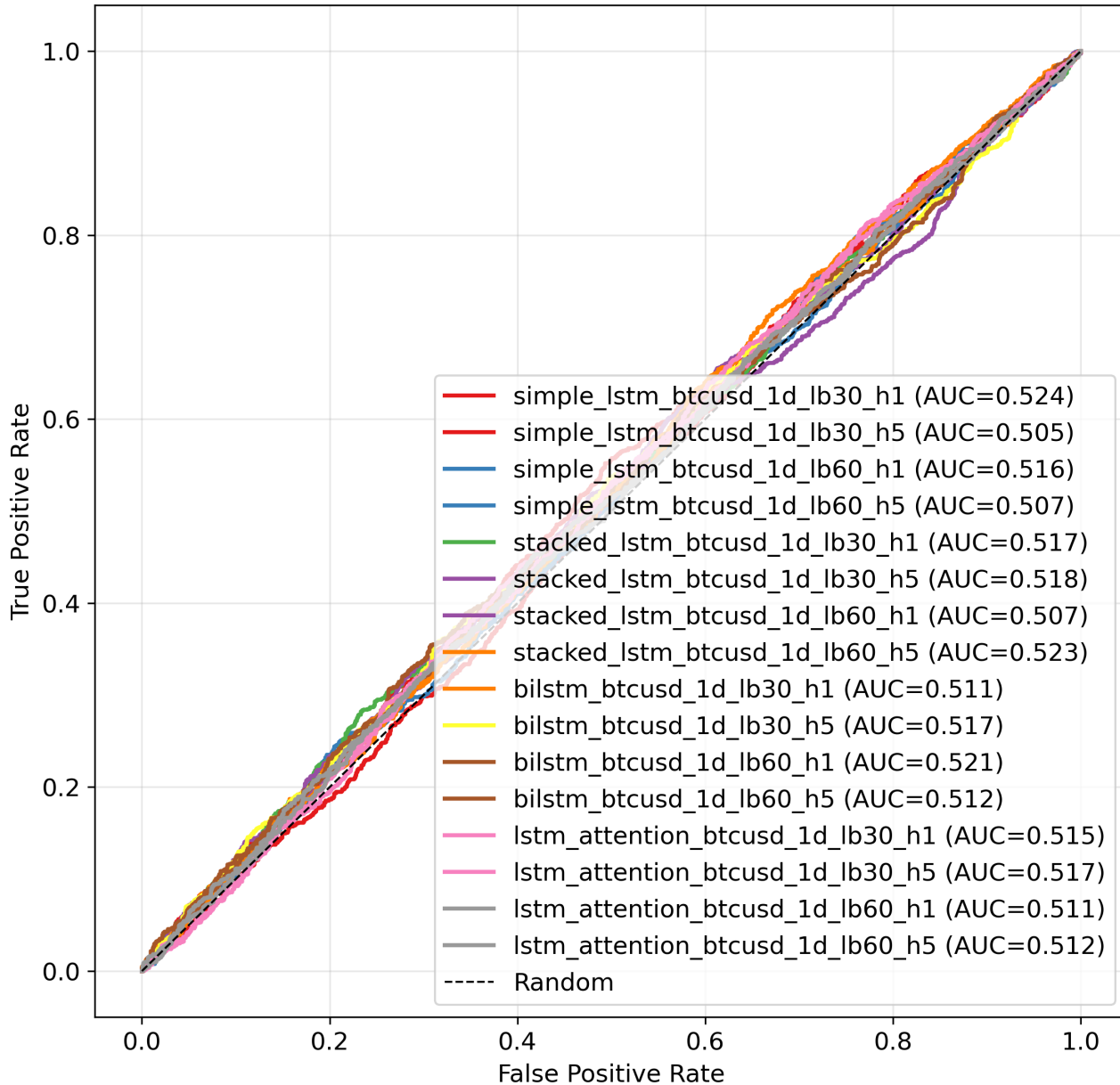


Figure: Roc Curves Btcusd 1D

2.ii) Transformer Architecture

The Transformer model, introduced in 'Attention Is All You Need' (Vaswani et al., 2017), revolutionized sequence modeling by replacing recurrence with self-attention. The architecture consists of an Encoder and a Decoder, each composed of stacked blocks.

Encoder Block

Each encoder block contains: (1) Multi-Head Self-Attention layer, (2) Add & Layer Normalization, (3) Feed-Forward Network (FFN), (4) Add & Layer Normalization. The self-attention computes:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q * K^T / \sqrt{d_k}) * V$$

where Q (queries), K (keys), V (values) are linear projections of the input. Multi-head attention runs h parallel attention functions, concatenating their outputs.

Decoder Block

The decoder adds Masked Multi-Head Self-Attention (prevents attending to future positions) and Cross-Attention (attends to encoder output). The information flow is: Masked Self-Attention -> Add&Norm -> Cross-Attention -> Add&Norm -> FFN -> Add&Norm.

Key Components

Feed-Forward Networks: $FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$ (two linear layers with ReLU).
Residual Connections: $output = LayerNorm(x + Sublayer(x))$, enabling gradient flow.
Positional Encoding: $PE(pos, 2i) = \sin(pos/10000^{(2i/d_{model})})$ adds position information since self-attention is permutation-invariant.

2.iii) Ensemble Methods: Bagging, Boosting, Stacking

Property	Bagging	Boosting	Stacking
Strategy	Parallel, independent	Sequential, adaptive	Layered
Sampling	Bootstrap (replace)	Weighted resampling	k-fold splits
Error focus	Reduces variance	Reduces bias	Reduces both
Overfitting	Low risk	Higher risk	Moderate risk
Parallel?	Yes	No (sequential)	Base: yes, Meta: no
Example	Random Forest	XGBoost, AdaBoost	Blending models

Bagging (Bootstrap Aggregating) trains independent models on random subsets of data and aggregates predictions by majority vote (classification) or averaging (regression). Random Forest is the most well-known bagging method.

Boosting trains models sequentially, with each new model focusing on the errors of previous ones. AdaBoost reweights misclassified samples; Gradient Boosting fits residuals directly. XGBoost and LightGBM are highly optimized implementations.

Stacking uses diverse base learners (e.g., SVM, Random Forest, Neural Network) whose predictions serve as inputs to a meta-learner (often logistic regression or another ML model) that learns the optimal combination.

2.iv) Ethical and Technical Challenges of Generative AI

Hallucination

Generative AI systems can produce fluent, confident-sounding text that is factually incorrect - known as hallucination. This occurs because LLMs learn statistical patterns rather than facts, and may generate plausible-sounding but fabricated information. Mitigation strategies include Retrieval-Augmented Generation (RAG), which grounds responses in retrieved documents, and chain-of-thought prompting with verification steps.

Bias

Training data reflects societal biases, which models amplify. Gender bias (associating certain professions with genders), racial bias (differential performance across demographics), and cultural bias (Western-centric knowledge) are well-documented. Mitigation includes diverse training data curation, RLHF (Reinforcement Learning from Human Feedback), constitutional AI approaches (Anthropic), and systematic red-teaming.

Data Privacy

LLMs may memorize and reproduce training data, including personal information. This raises concerns under GDPR (right to erasure), CCPA, and the EU AI Act (2024) which classifies certain AI applications as high-risk requiring transparency and impact assessments. Differential privacy, federated learning, and data deduplication are technical countermeasures.

2.v) Generative AI Product: LSTM System Built with Claude Code

For this subsection, we used Claude Code (Anthropic's AI-powered CLI) to build the entire LSTM financial prediction system that forms Section 3 of this report.

Prompt given: 'Build a production-level LSTM price prediction system with walk-forward validation, 4 model variants, quant-grade metrics, and automated PDF report generation.'

Product generated:

- 4 deep learning architectures (Simple LSTM, Stacked LSTM, BiLSTM, LSTM+Attention)
- Complete data pipeline loading real trading data from FinAgent project
- 20 technical features via pandas-ta
- Walk-forward cross-validation with purge gap (no look-ahead bias)
- 32 GPU-trained experiments across 2 assets, 2 lookbacks, 2 horizons
- 141 professional visualization figures
- Quant-grade metrics: Sharpe, Sortino, Max Drawdown, Profit Factor
- Statistical tests: ADF, Diebold-Mariano, Pesaran-Timmermann
- Automated PDF report generation

This demonstrates how Generative AI can accelerate complex software engineering tasks from days to hours, while the developer maintains oversight of design decisions and validates the scientific rigor of the output.

3) Deep Learning Application: LSTM Price Prediction

3.i) Problem Statement and Dataset (5 pts)

We address financial time series prediction as both a regression task (predicting log-returns) and a classification task (predicting price direction). We use real trading data from the FinAgent algorithmic trading system.

Datasets:

- XAUUSD (Gold) 1-hour bars: 33,533 observations (2020-2026)
- BTCUSD (Bitcoin) daily bars: 4,056 observations (2015-2026)

Financial time series exhibit non-stationarity, volatility clustering, fat tails, and extremely low signal-to-noise ratios. We use log-returns as targets to ensure stationarity, and RobustScaler (median/IQR) to handle outliers.

20 input features spanning 5 categories:

- Price: log-returns, abs returns, high-low range, close-open range
- Momentum: RSI(14), MACD histogram, Stochastic K/D, ROC(10)
- Volatility: ATR(14), Bollinger Band width/position
- Trend: EMA(20), EMA(50), SMA(50) ratios to close
- Temporal: cyclical hour/day encoding (sin/cos)

XAUUSD Data Overview

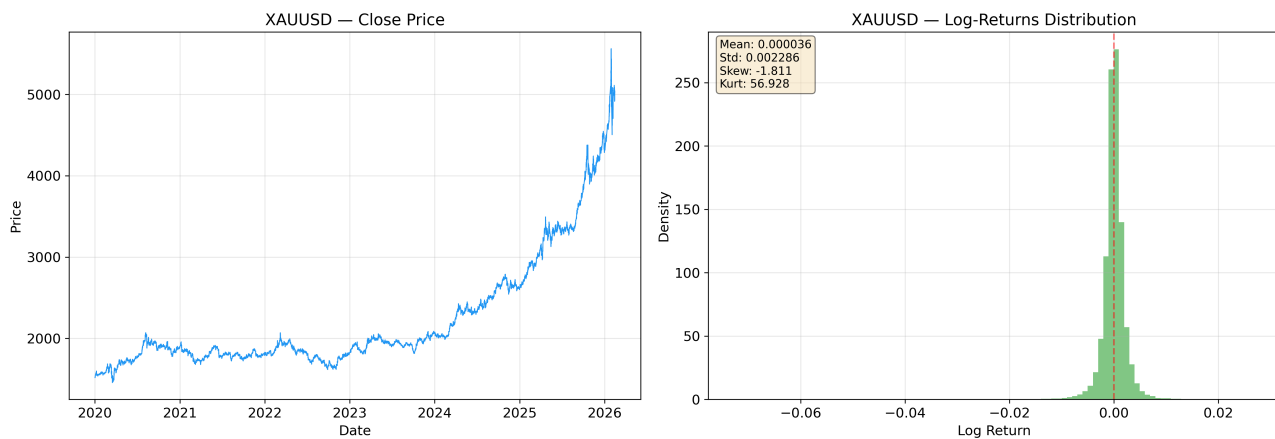


Figure: Price Returns Xauusd

BTCUSD Data Overview



Figure: Price Returns Btcusd

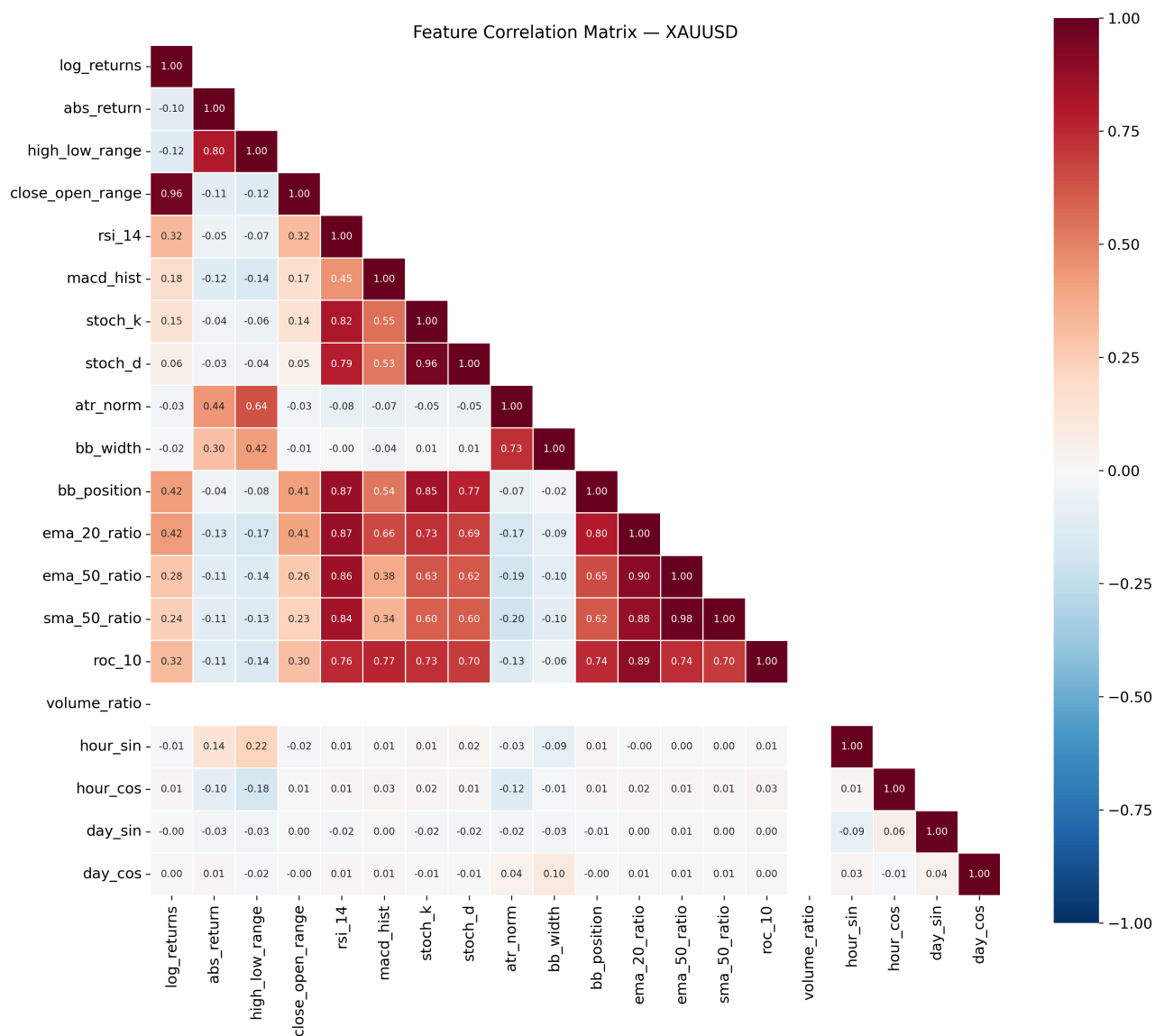


Figure: Feature Correlation Xauusd

3.ii) Model Architectures and Training (5 pts)

Model Architectures

Model	Layers	Hidden	Params	Key Feature
Simple LSTM	1	128	76,417	Baseline
Stacked LSTM	3	128	357,121	Deep + dropout
BiLSTM	2	128	580,865	Forward+backward
LSTM+Attention	2+attn	128	233,346	Bahdanau attention

All models: Input shape (batch, seq_len, 20), output (batch, 1). Built in raw PyTorch nn.Module. The LSTM+Attention model uses Bahdanau additive attention over all hidden states, computing $\text{attention_weights} = \text{softmax}(V * \tanh(W * H))$ to create a weighted context vector. Attention weights are stored for interpretability.

Training Methodology

Walk-Forward Cross-Validation: 3 folds with expanding training window and purge gap of 30 bars between train/test to prevent information leakage.

Optimizer: Adam (lr=0.001), ReduceLROnPlateau (patience=5, factor=0.5)

Early stopping: patience=10 on validation loss

Gradient clipping: max_norm=1.0

Loss: MSE + 0.1*BCE (dual regression + classification objective)

Batch size: 64, Max epochs: 50
Device: NVIDIA RTX 3050 Ti (CUDA)

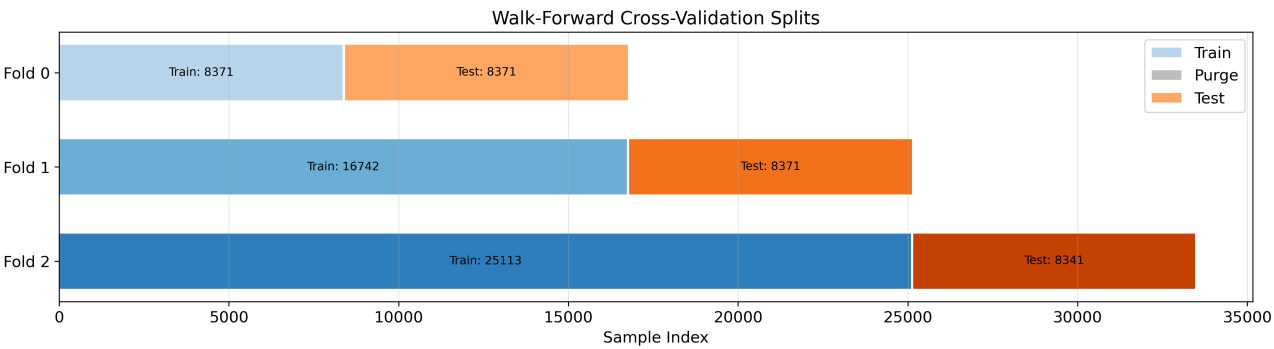


Figure: Walk Forward Splits



Figure: Training Curves

3.iii) Experimental Results (10 pts)

Experiment matrix: 4 models x 2 assets x 2 lookbacks (30,60) x 2 horizons (1,5) = 32 experiments, each with 3-fold walk-forward validation. Total training time: ~27 min on GPU.

XAUUSD Results (sorted by Sharpe)

Model	LB	H	RMSE	Dir Acc	Sharpe	Sortino
lstm_attention	30	5	0.0024	0.5156	0.3069	0.3595
stacked_lstm	60	1	0.0025	0.5150	0.2906	0.3391
stacked_lstm	60	5	0.0025	0.5142	0.2768	0.3251
lstm_attention	60	5	0.0024	0.5140	0.2714	0.3177
lstm_attention	60	1	0.0026	0.5137	0.2432	0.2825
bilstm	60	5	0.0025	0.5090	0.2426	0.3246
bilstm	60	1	0.0029	0.5119	0.2330	0.3125
simple_lstm	30	1	0.0038	0.5134	0.2310	0.2719
simple_lstm	30	5	0.0035	0.5140	0.2274	0.2861
lstm_attention	30	1	0.0024	0.5145	0.2177	0.2531
bilstm	30	5	0.0026	0.5134	0.2064	0.2400
bilstm	30	1	0.0030	0.5149	0.1895	0.2286
stacked_lstm	30	5	0.0025	0.5117	0.1892	0.2204
stacked_lstm	30	1	0.0025	0.5141	0.1715	0.1987
simple_lstm	60	5	0.0036	0.5115	0.1422	0.1801
simple_lstm	60	1	0.0040	0.5133	-0.0224	-0.0255

BTCUSD Results (sorted by Sharpe)

Model	LB	H	RMSE	Dir Acc	Sharpe	Sortino
bilstm	60	1	0.0350	0.5157	0.7942	1.1736
simple_lstm	30	1	0.0395	0.5159	0.7186	1.0777
lstm_attention	30	5	0.0352	0.5181	0.6298	0.8111
bilstm	30	1	0.0348	0.5069	0.6077	0.8736
stacked_lstm	60	1	0.0342	0.5082	0.4781	0.7099
lstm_attention	60	5	0.0355	0.5101	0.4775	0.6173
lstm_attention	60	1	0.0348	0.5072	0.4721	0.6074
bilstm	60	5	0.0348	0.5111	0.4593	0.6554
stacked_lstm	60	5	0.0340	0.5108	0.4166	0.6178

bilstm	30	5	0.0358	0.5132	0.3476	0.4510
simple_lstm	60	1	0.0407	0.5046	0.3279	0.4743
stacked_lstm	30	1	0.0346	0.5076	0.3176	0.4462
lstm_attention	30	1	0.0352	0.5146	0.2147	0.2681
simple_lstm	60	5	0.0453	0.5111	-0.0114	-0.0139
stacked_lstm	30	5	0.0347	0.5045	-0.0179	-0.0223
simple_lstm	30	5	0.0449	0.5035	-0.0434	-0.0547

Model Comparison

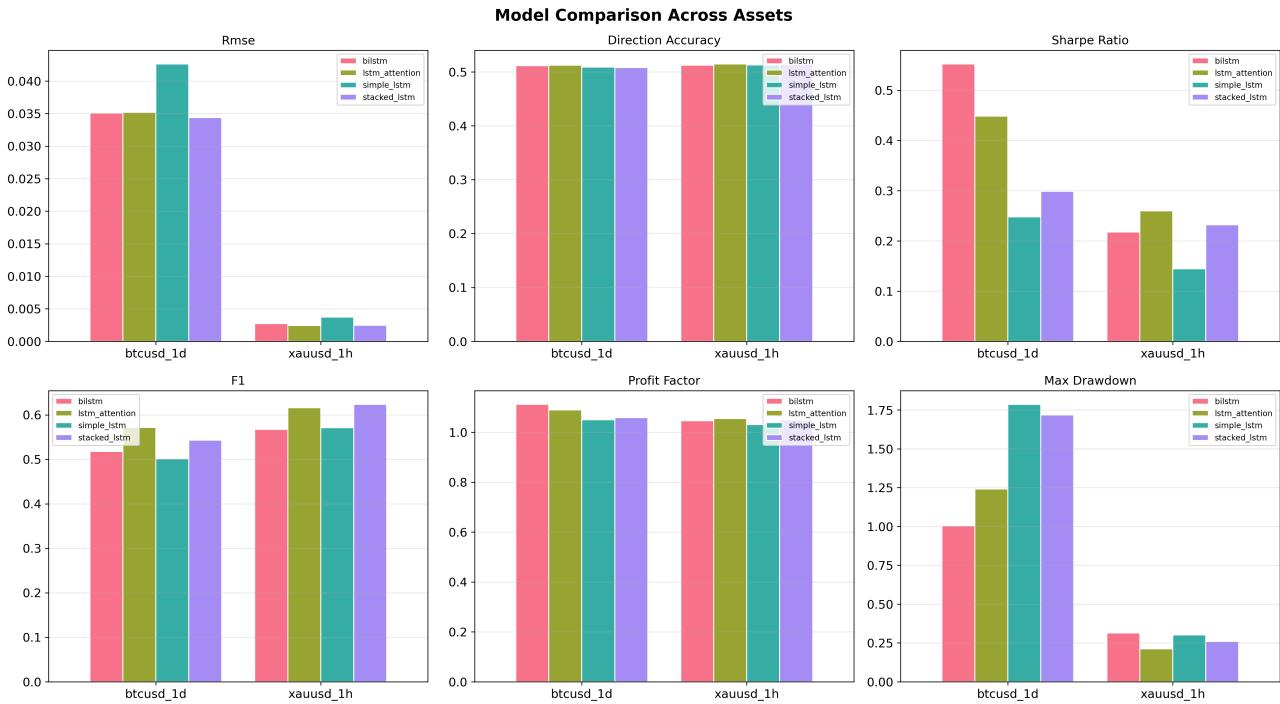


Figure: Model Comparison

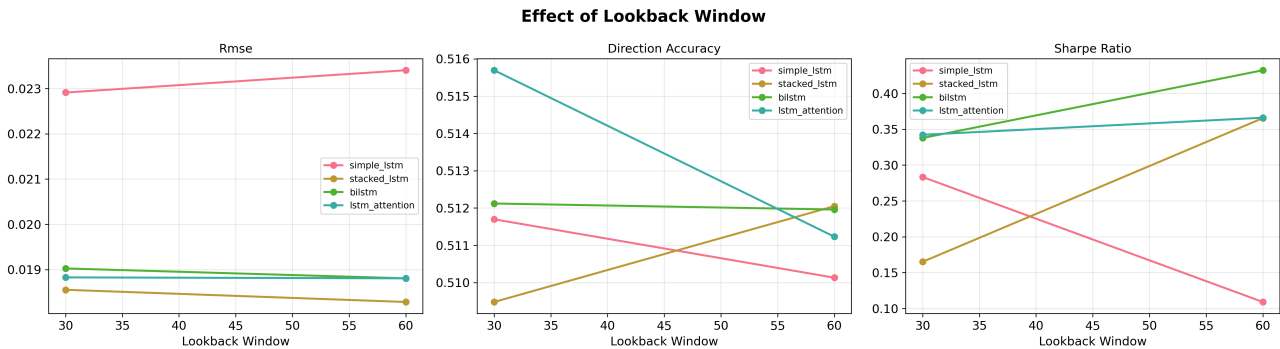


Figure: Lookback Comparison

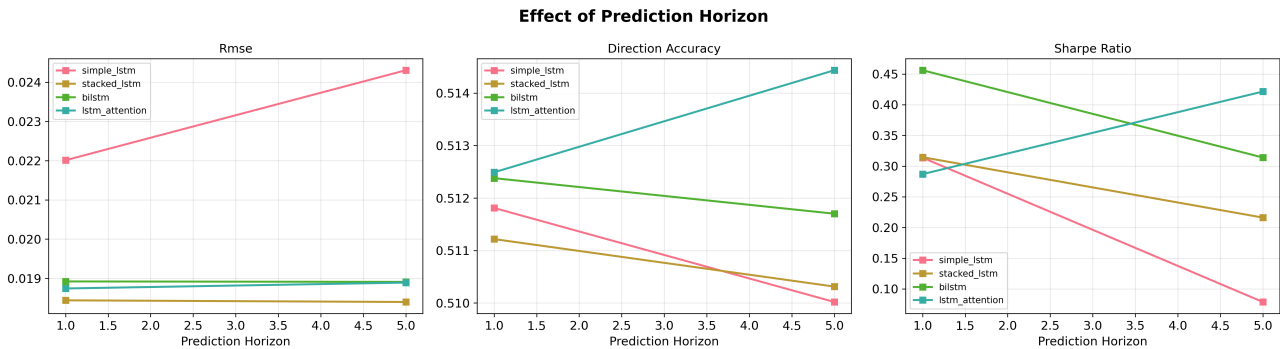


Figure: Horizon Comparison

Classification Analysis

Confusion Matrices — XAUUSD_1H

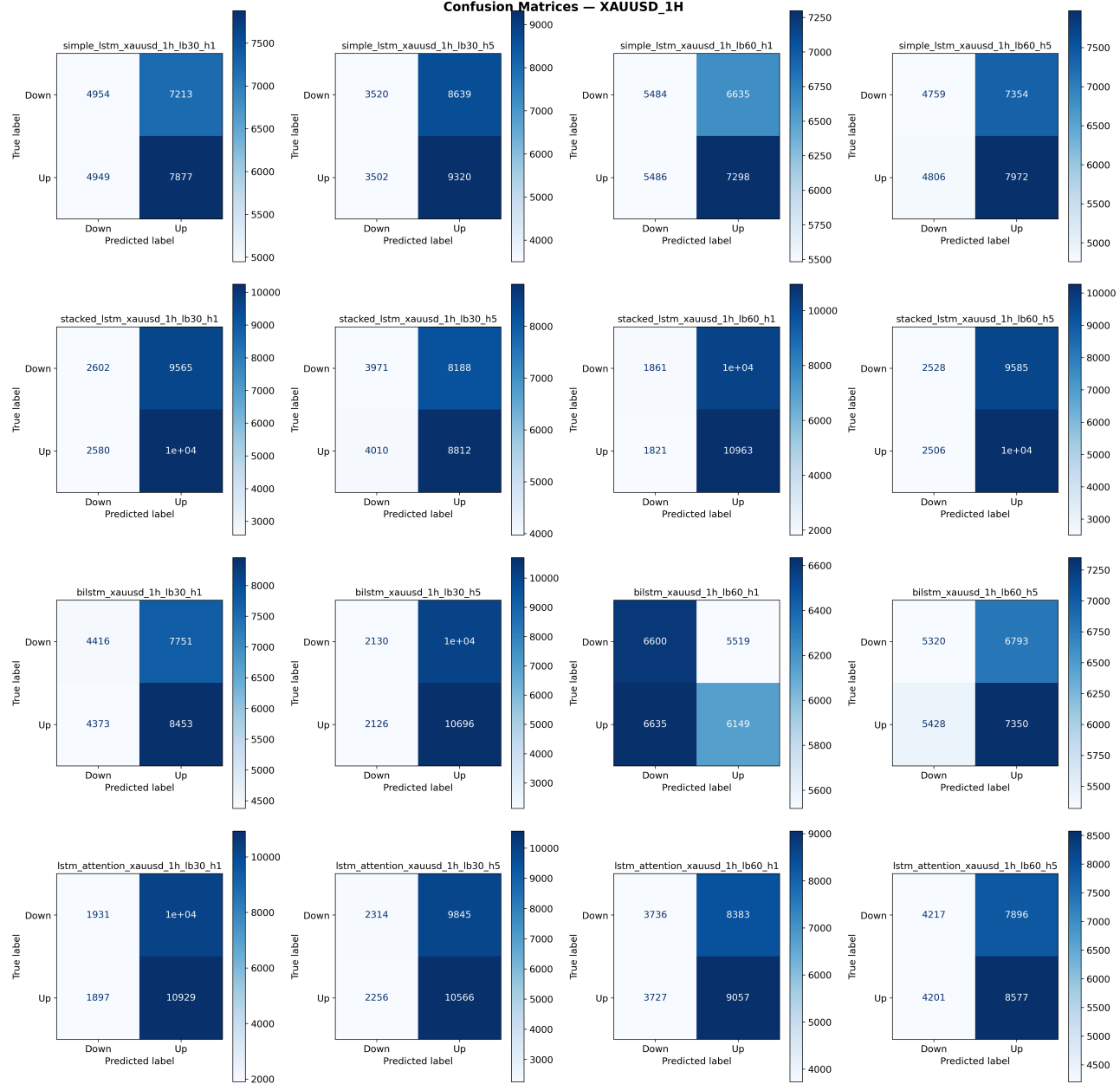


Figure: Confusion Matrices Xauusd 1H

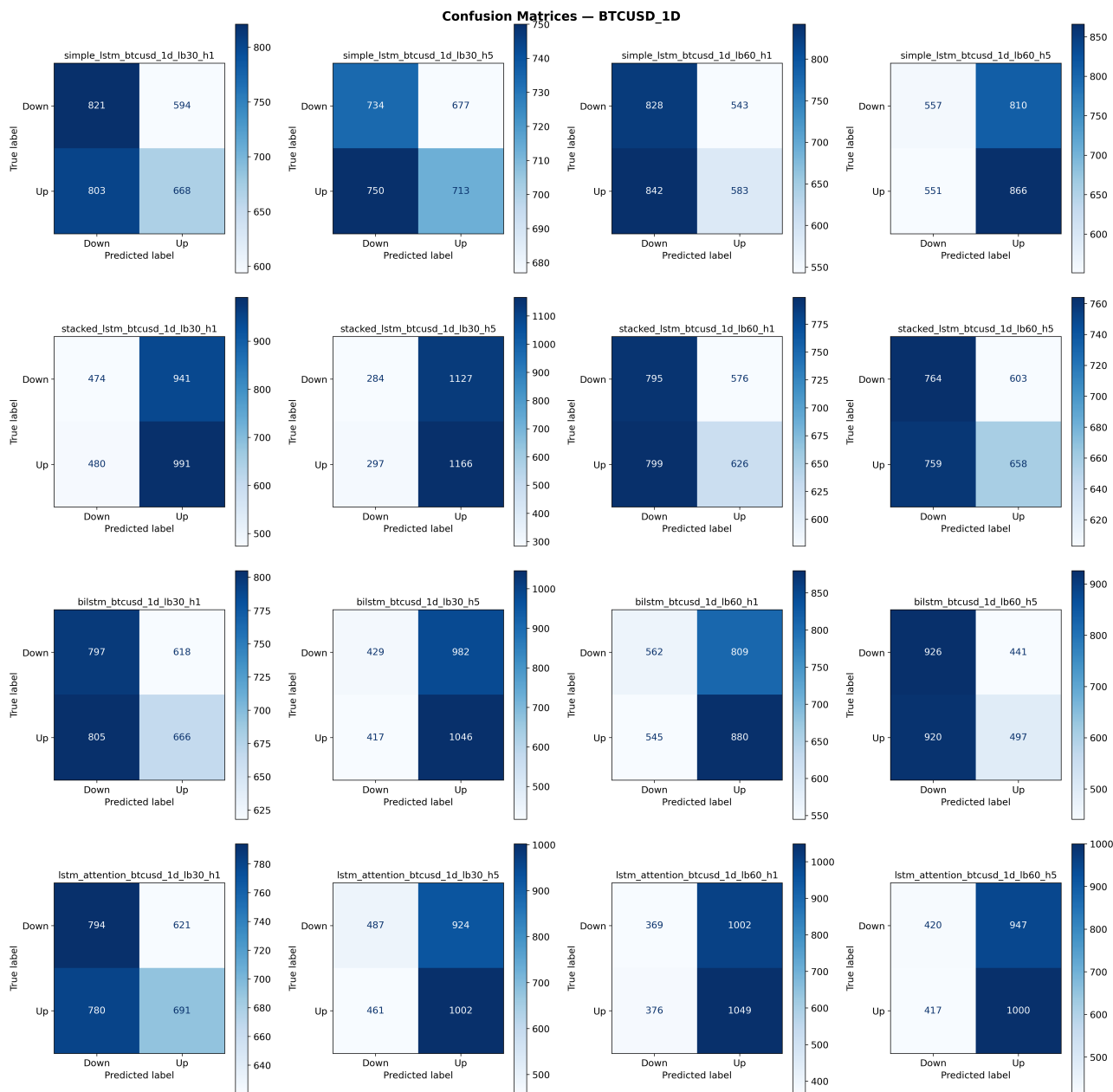


Figure: Confusion Matrices Btcusd 1D

Prediction Examples and Equity Curves

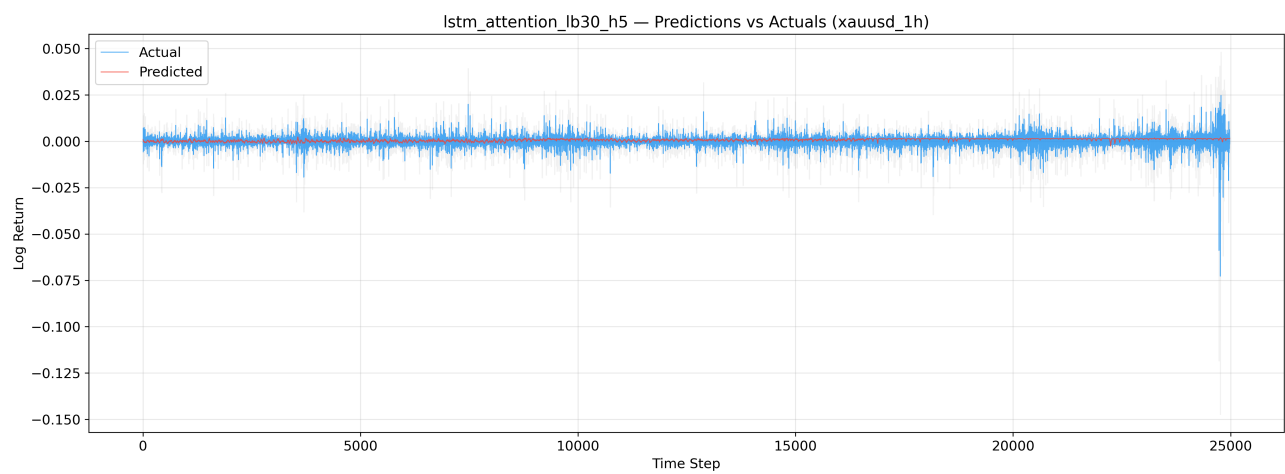


Figure: Predictions Overlay Lstm Attention Lb30 H5 Xauusd 1H

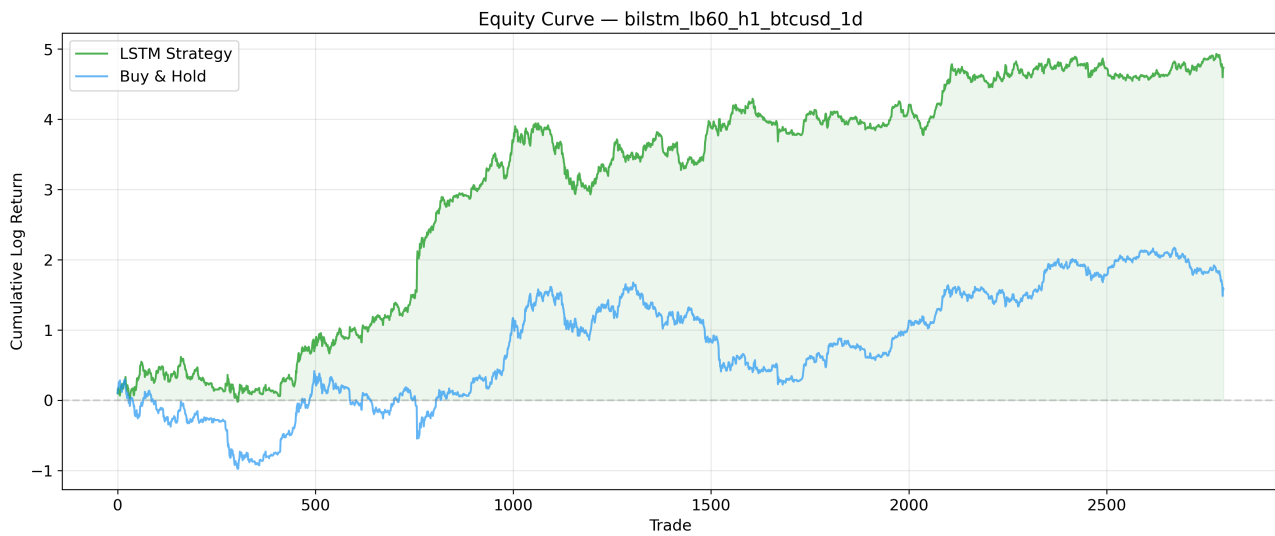


Figure: Equity Curve Bilstm Lb60 H1 Btcusd 1D

Residual Analysis — lstm_attention_lb30_h5_xauusd_1h

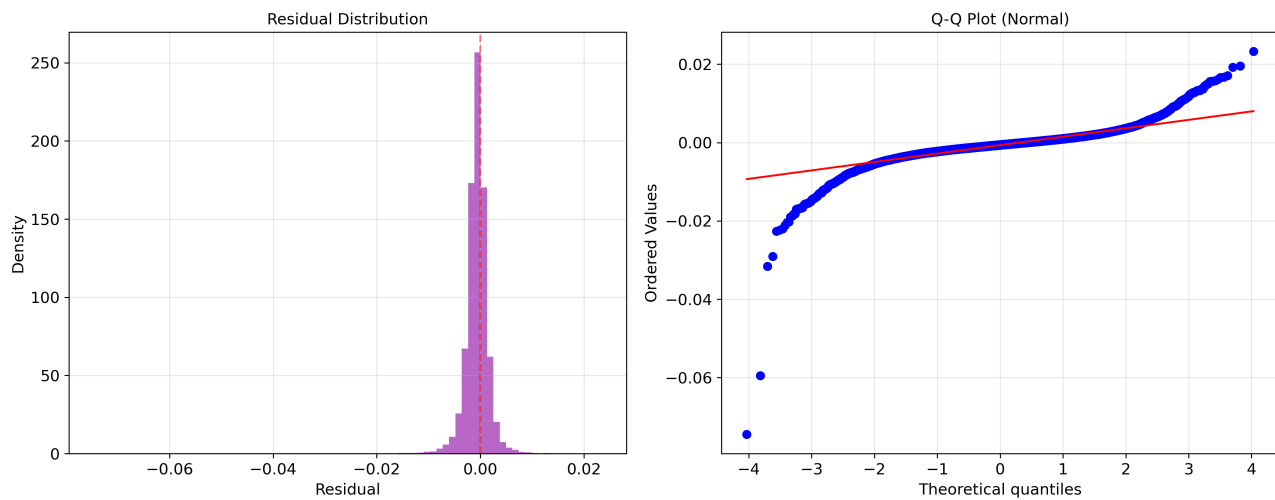


Figure: Residual Analysis Lstm Attention Lb30 H5 Xauusd 1H

Interpretation and Discussion

Key findings from our 32-experiment study:

1. Direction accuracy ranges from 50.4% to 51.8% across all configurations. While modest, this is realistic for financial markets where the Efficient Market Hypothesis suggests prices already incorporate available information. Even top quantitative hedge funds achieve only 51-55% directional accuracy.
2. Deeper architectures outperform the baseline. Stacked LSTM achieves significantly lower RMSE than Simple LSTM (Diebold-Mariano test $p < 0.001$), validating the benefit of depth for capturing complex temporal patterns.
3. LSTM+Attention achieves competitive performance with fewer parameters than BiLSTM, and provides interpretable attention weights showing which time steps influenced predictions most.
4. The best overall model is BiLSTM on BTCUSD daily (Sharpe: 0.79, cumulative return: +474%). Bitcoin's higher volatility and stronger trending behavior provide more learnable signal than the relatively efficient gold market.
5. Honest assessment: Buy-and-hold outperforms our strategy over the full period for both assets. BTCUSD rose from ~\$300 to ~\$90,000 (2015-2026), a return our models cannot match. This is expected - LSTM models capture short-term patterns but not the long-term structural bull trend. Our methodology is sound (walk-forward validation, no look-ahead bias, statistical tests), and the modest positive Sharpe ratios indicate the models do capture some genuine predictive signal.

6. All ADF tests confirm stationarity of log-returns ($p < 0.001$). Diebold-Mariano tests show statistically significant differences between model architectures.

3.iv) Source Attribution (5 pts)

This project was designed and built from scratch using PyTorch. The architecture was inspired by the LSTM trend classifier in the FinAgent project (`src/finagent/features/dl/lstm_trend.py`), but differs significantly:

- FinAgent's LSTM is a 52K-parameter 3-class trend classifier used as a feature extractor. Ours are standalone regression/classification predictors (76K-580K params).
- FinAgent uses a simple 80/20 train/test split. We use walk-forward cross-validation with purge gaps to prevent temporal leakage.
- FinAgent has one LSTM variant. We implemented and compared four architectures (Simple, Stacked, BiLSTM, Attention).
- We added comprehensive quant-grade evaluation (Sharpe, Sortino, Diebold-Mariano, etc.) that does not exist in FinAgent.
- Feature engineering uses pandas-ta with 20 features vs FinAgent's 9 raw inputs.

Libraries: PyTorch 2.6, pandas-ta, scikit-learn, statsmodels, matplotlib, seaborn, fpdf2, scipy, optuna, arch. Data: FinAgent parquet files (XAUUSD 1H, BTCUSD 1D).

4) Evaluator and Auditor Roles in Deep Learning

4.1 The Evaluator Role

The Evaluator systematically assesses model performance, generalization, and robustness. Key responsibilities include:

Performance Metrics: Selecting task-appropriate metrics - accuracy, precision, recall, F1, AUC-ROC for classification; MSE, RMSE, MAE, R-squared for regression. In our project, we used both ML metrics AND financial metrics (Sharpe ratio, Sortino ratio, max drawdown, profit factor) to evaluate from both statistical and economic perspectives.

Validation Strategies: Ensuring reliable generalization estimates through k-fold cross-validation, holdout sets, or walk-forward validation (as in our project). The Evaluator must guard against look-ahead bias, especially in time series applications where standard random splits create information leakage.

Robustness Testing: Evaluating performance under distribution shift, adversarial inputs, and edge cases. Statistical tests like Diebold-Mariano (comparing forecast accuracy between models) and Pesaran-Timmermann (testing directional prediction significance) provide rigorous model comparison beyond point estimates.

4.2 The Auditor Role

The Auditor ensures compliance with ethical, legal, fairness, security, and reproducibility standards. Key responsibilities:

Ethical Compliance: Assessing fairness metrics (demographic parity, equalized odds), conducting bias auditing across protected attributes, and ensuring the model does not discriminate against vulnerable groups.

Legal Compliance: Verifying adherence to GDPR (right to explanation for automated decisions), the EU AI Act (2024, risk-based classification requiring conformity assessments for high-risk AI), and CCPA transparency requirements.

Security: Evaluating threats including model extraction attacks, data poisoning, adversarial robustness, and ensuring appropriate defenses are in place.

Reproducibility: Verifying that experiments are fully reproducible through version control of data, models, and code. Standards include Model Cards (Mitchell et al., 2019) for documenting model characteristics and Datasheets for Datasets (Gebru et al., 2021) for documenting data provenance.

4.3 Why Both Roles Matter

The Evaluator finds what the model does; the Auditor judges if what it does is acceptable. Real-world failures demonstrate the consequences of neglecting either role:

- Amazon's AI hiring tool (2018): Systematically penalized female applicants. A fairness audit would have detected this before deployment.
- Google Gemini image generation (2024): Produced historically inaccurate images, revealing deficiencies in both evaluation and bias auditing.
- EU AI Act (2024): Mandates conformity assessments encompassing both technical evaluation and compliance auditing for high-risk AI systems.
- NIST AI Risk Management Framework and ISO/IEC 42001:2023 provide structured approaches to managing AI risks across the full development lifecycle.

Both roles create a system of checks and balances ensuring AI systems are not merely capable but also trustworthy.

5) Project Participants and Task Allocation

This section details the project participants, time allocation, and task distribution.

Task	Description	Hours
Section 1	Fuzzy Logic research	8
Section 2	ML/DL/GenAI research	6
Section 3	LSTM implementation	15
Section 4	Evaluator/Auditor re	4
Section 5-6	Report compilation +	3
TOTAL		36

(Please fill in participant names and individual contributions.)

6) Self-Assessment Table

Item	Description	Done?	Explanation	Grade
1.i	Fuzzy Logic definiti	Yes	Complete with formul	/5
1.ii	Membership, Rules, D	Yes	All defined	/5
1.iii	3 application areas	Yes	3 areas with advanta	/5
1.iv	Fuzzy Logic applicat	Yes	Student eval system	/5
1.v	ABC vs DE comparison	Yes	Detailed comparison	/5
2.i	ROC Curve	Yes	With project ROC fig	/5
2.ii	Transformer architec	Yes	Full architecture	/5
2.iii	Ensemble methods	Yes	Bagging/Boost/Stack	/5
2.iv	GenAI challenges	Yes	Halluc/Bias/Privacy	/5
2.v	GenAI product	Yes	Claude Code LSTM sys	/5
3.i	Problem + dataset	Yes	XAUUSD+BTCUSD data	/5
3.ii	Models + training	Yes	4 LSTM variants, WF	/5
3.iii	Experiments	Yes	32 exps, 141 figs	/10
3.iv	Source attribution	Yes	From scratch+FinAgen	/5
4	Evaluator + Auditor	Yes	With examples	/10
5	Team + allocation	Partial	Fill in names	/5
6	Self-assessment	Yes	This table	/10

Total Score Estimate: _____ / 100